MCFuser: High-Performance and Rapid Fusion of Memory-Bound Compute-Intensive Operators

Zheng Zhang
School of Computer Science
Wuhan University
zzhang3031@whu.edu.cn

Donglin Yang
NVIDIA
dongliny@nvidia.com

Xiaobo Zhou
University of Macau
waynexzhou@um.edu.mo

Dazhao Cheng*
School of Computer Science
Wuhan University
dcheng@whu.edu.cn

***Abstract*—Operator fusion, a key technique to improve data locality and alleviate GPU memory bandwidth pressure, often fails to extend to the fusion of multiple compute-intensive operators due to saturated computation throughput. However, the dynamicity of tensor dimension sizes could potentially lead to these operators becoming memory-bound, necessitating the generation of fused kernels — a task hindered by limited search spaces for fusion strategies, redundant memory access, and prolonged tuning time, leading to sub-optimal performance and inefficient deployment.**

We introduce MCFuser, a pioneering framework designed to overcome these obstacles by generating high-performance fused kernels for what we define as memory-bound compute-intensive (MBCI) operator chains. Leveraging high-level tiling expressions to delineate a comprehensive search space, coupled with Directed Acyclic Graph (DAG) analysis to eliminate redundant memory accesses, MCFuser streamlines kernel optimization. By implementing guidelines to prune the search space and incorporating an analytical performance model with a heuristic search, MCFuser not only significantly accelerates the tuning process but also demonstrates superior performance. Benchmarked against leading compilers like Ansor on NVIDIA A100 and RTX3080 GPUs, MCFuser achieves up to a 5.9x speedup in kernel performance and outpaces other baselines while reducing tuning time by over 70-fold, showcasing its agility.

***Index Terms*—Operator Fusion, Compute-Intensive, Automatic Exploration, Performance Model, GPU**

I. Introduction

As deep learning models grow in size and complexity, they necessitate a myriad of operators, which in turn significantly burdens GPU memory bandwidth with frequent I/O operators [1]–[4]. Operator fusion emerges as a pivotal strategy to mitigate this challenge [5]–[7], aiming to diminish kernel launch overhead and improve data locality by curtailing the reliance on frequent global memory access. It has been widely embraced across various libraries [8]–[12] and tensor program compilers [13]–[21] to enhance runtime performance by consolidating operators, thereby optimizing computational efficiency. Frameworks such as Cutlass [10], TensorRT [11], and high-level compilers like TVM [14], [22] and Ansor [13], have deployed operator fusion to address computational and memory bandwidth constraints.

Deep learning operators typically fall into one of two categories: compute-intensive, like GEMM, or memory-intensive, such as LayerNorm and Softmax. The latter often saturates GPU memory bandwidth due to their frequent memory accesses, resulting in the under-utilization of computing resources. To address this, several studies have explored operator fusion, either among memory-intensive operators or between memory-intensive operators and a single compute-intensive operator, aiming to optimize resource use. Specifically, AStitch [7] focuses solely on fusing memory-intensive operators, while DNNFusion [6] fuses a compute-intensive operator with its adjacent memory-intensive operators. Despite these efforts, the fusion of compute-intensive operator chains, which could further exploit computing resources, remains largely unaddressed. This oversight reveals a significant opportunity for enhancing computational performance beyond the current scope of most deep-learning compilers.

The distinction between computation and memory bandwidth bottlenecks in deep learning operators is not solely dependent on the operator type but also on input tensor characteristics. This is demonstrated by how dynamic changes in tensor dimensions can shift an operator's nature. For example, in a GEMM operator $C_{M,N} = A_{M,K} \times B_{K,N}$, altering the dimension K from 1,024 to 1 on modern accelerators can dramatically change the computation to memory access ratio from 227 to 2, transitioning the operator from compute-bound to memory-bound. In scenarios where reduction dimensions are minimized, traditionally compute-intensive operators can unexpectedly encounter memory bandwidth limitations, transforming them into memory-bound scenarios. This observation leads us to identify a unique category of operators, termed memory-bound compute-intensive (MBCI) operators, which exhibit this dual nature under certain conditions.

MBCI operators, exemplified by self-attention modules in transformative architectures like the Transformer [23], underscore a critical optimization frontier in deep learning. For instance, in the BERT-base model processing a 1,024-sequence input, self-attention constitutes only 14% of the computational workload (FLOPs) but dominates 51% of the execution time, revealing substantial optimization potential. Despite the promising avenue for performance enhancement through kernel fusion, represented typically by hierarchical loop structures and computation-memory access patterns, several obstacles impede the efficient fusion of MBCI operator chains. Firstly, the search space for fusion strategies often

*Corresponding author.

SC24, November 17-22, 2024, Atlanta, Georgia, USA
979-8-3503-5291-7/24/$31.00 ©2024 IEEE

remains incomplete; for example, Chimera [5] limits itself to nested loop computations, neglecting alternate execution sequences within the same loop scope. Secondly, the direct coupling of memory accesses with computation loops can lead to redundant data movements, particularly noticeable in cases where loop extents are minimized, as seen in Chimera's implementation. Lastly, the fusion process is frequently bogged down by prolonged auto-tuning phases and unwieldy search spaces. Frameworks like Ansor require hours for operator optimization through ML-guided exploration (e.g., XGBoost [24]), relying on extensive runtime statistics for model training and significantly increasing the tuning overhead.

To overcome the outlined challenges in fusing MBCI operator chains, we introduce MCFuser, a framework designed for high performance and efficiency. MCFuser starts with constructing a comprehensive search space using high-level tiling expressions, enabling the complete exploration of potential fused operator decompositions. By leveraging DAG analysis, we strategically decouple memory access statements from loops, significantly reducing redundant memory transfers between GPU global and shared memory. To streamline the search process and mitigate time expenditures, MCFuser employs heuristic guidelines for pruning non-viable segments of the search space. A pivotal innovation of our framework is the development of an analytical performance model based on an exhaustive analysis of computational and memory transactions. This model dramatically cuts down the need for extensive program measurement and model training. Incorporating this performance model into a heuristic search mechanism, MCFuser automatically identifies the optimal configuration for fused tensor programs, marking a significant leap forward in the efficient optimization of MBCI operators.

In summary, our contributions are as follows:

- We identify opportunities for enhancing data locality in tensors by analyzing compute-intensive operator bottlenecks across varying input tensor dimensions, which lead to the fusion of MBCI operators.
- We construct a comprehensive search space for MBCI operator fusion through detailed tiling expressions and reducing redundant memory access via DAG analysis.
- We develop efficient pruning guidelines and integrate an analytical performance model with a heuristic search method, which collectively streamlines the tuning process, enabling automatic and efficient identification of the optimal candidate in the search space.

We encapsulate our innovations in MCFuser and rigorously evaluate its performance across diverse workloads on NVIDIA GPUs. MCFuser's tensor programs significantly surpass existing methods, achieving up to a 5.9× speedup over Ansor [13] and consistently outperforming competitors like BOLT [25] and PyTorch [26]. Moreover, it dramatically reduces tuning times by more than 70 times, underscoring MCFuser's efficiency and effectiveness in optimizing tensor operators.

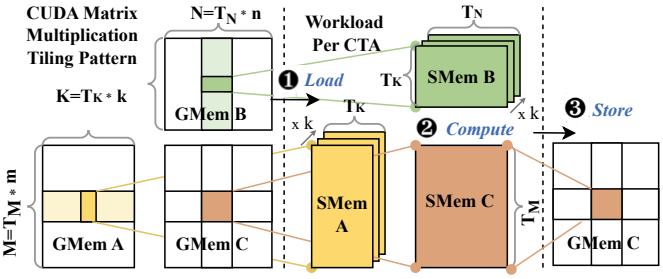


Fig. 1. Illustration of the Matmul operator implemented on the CUDA platform, indicating *GMem* for global memory and *SMem* for shared memory.

II. Background and Motivation

A. Operator Fusion Characteristics

Operator fusion, acclaimed for reducing kernel launch and memory access overheads, categorizes tensor operators into compute-intensive and memory-intensive types. Compute-intensive operators, such as GEMM, demand high computational resources, whereas memory-intensive operators, like ReLU and softmax, are limited by memory bandwidth [5], [7]. This classification underlies optimization strategies, notably epilogue fusion, which merges a compute-intensive operator with the following memory-intensive ones, illustrated by the coupling of MatMul and ReLU [10], [27], [28]. These practices aim to balance computational demands with memory efficiency.

However, while compute-intensive operators are generally characterized by high computational intensity, they can become memory-bound as tensor dimensions change. Consider the MatMul ($C = A \times B + C$) operator as an example. Illustrated in Fig. 1, each thread block computes a tile of C, initially loading tiles from global memory to shared memory ❶, then accumulating partial results of tile C over k iterations ❷, and finally writing the result back to global memory ❸.

The theoretical ratio of compute operations to memory accesses, denoted as $\phi = (2T_M T_N K)/(2T_M T_N + T_M K + T_N K)$, serves as a critical indicator of performance bottlenecks. As illustrated in Fig.2, a decrease in the K/M ratio leads to a higher proportion of memory accesses relative to compute operations, making the tensor program increasingly memory bandwidth-bound and diminishing computational throughput. This dynamic reveals that the compute-intensive operator, MatMul, transitions to a memory-bound state when $\phi < \mathcal{P}/\mathcal{W}$, with $\mathcal{P}$ being the peak computation throughput and $\mathcal{W}$ the memory bandwidth, thus classifying it as an MBCI operator. Similar to memory-intensive operators, fusing MBCI operators improves data locality and mitigates memory bandwidth demands. However, traditional fusion strategies that rigidly classify compute-intensive operators as non-fusible boundaries fail to capitalize on the benefits of fusing MBCI operators, as highlighted by prior studies [6], [7], [29].

A Typical Case: Transformer models [23], pivotal in natural language processing and image classification, grapple with efficiency bottlenecks, especially in their self-attention modules,

TABLE I
THE COMPARISON AMONG EXISTING REPRESENTATIVE WORKS AND MCFUSER.

	Search Space			Memory Optimize	Exploration	
Name	**Support MBCI**	**Auto.**	**Search Space**	-	**Optimize Objective / Guidance**	**Tuning time**
AStitch [7]	No	Yes	Stitch Schemas	Fusion	Rule-based	Short
DNNFusion [6]	No	Yes	Pattern-based	Fusion	Mathematical Analyze	Short
BOLT [25]	Partial	Yes	Template-based	Fusion	Measured Performance	Mid
FlashAttention [30]	Partial	No	Handcrafted	Fusion	-	-
Ansor [13]	Yes	Yes	Loop Transformation	+ Loop Opt.	ML Cost Model(e.g. XGBoost [24])	Long
Chimera [5]	Yes	Yes	Nested Block Execution Order	+ Loop Opt.	Minimize Data Movement	Short
MCFuser (ours)	Yes	Yes	Exhaustive Tiling-based	+ Rid of Redundancy	Analytical Performance Model	Short

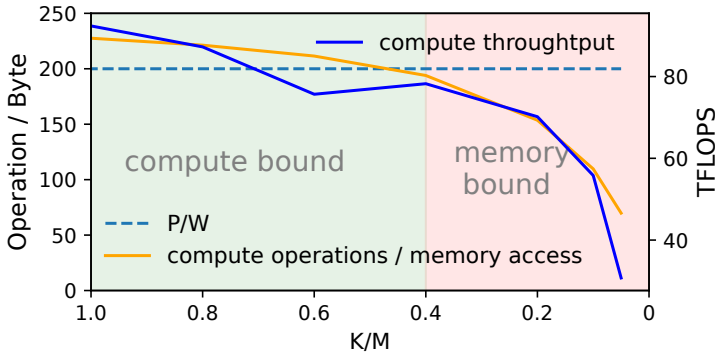


Fig. 2. The performance of a MatMul operator across different K/M ratios while maintaining constant computational complexity ($M \times N \times K = 1024^3$), where $M = N$). The theoretical ratio of compute operations to memory accesses for a tile size of 256 is represented on the left y-axis, while the actual computation throughput measured on the A100 GPU is shown on the right y-axis.

which exhibit quadratic time and memory complexity relative to sequence length [30]. For example, in the Bert-Large model by HuggingFace [31], despite self-attention modules contributing only 11%, 14%, and 19% of the FLOPs for sequence lengths of 512, 1024, and 2048, respectively, they disproportionately dominate execution time, consuming 39%, 51%, and 61% of it. This stark contrast highlights the self-attention module's inability to fully leverage computing resources, constrained significantly by memory bandwidth, and underscores the need for optimization strategies that address these specific inefficiencies.

B. Challenges and Limitations of Previous Works

In this section, we identify three primary challenges encountered in the development of efficient kernels for fusing MBCI operator chains. Furthermore, we discuss how existing literature falls short in addressing these challenges, with a comprehensive summary presented in Table I.

a) Incomplete Search Space: A fundamental challenge in fusing MBCI operator chains arises from an incomplete search space. For instance, AStitch [7] focuses on fusing traditional memory-intensive operators like element-wise, broadcast, and reduction operators, including LayerNorm and Softmax, overlooking the fusion potential for MBCI operators. Similarly, DNNFusion [6] merges compute-intensive operators with adjacent ones but refrains from directly fusing multiple compute-intensive operators, citing complexity. This limitation is echoed in several studies [15], [27], [29], which stereotype compute-intensive operators as non-fusible boundaries, missing opportunities for broader operator fusion. While efforts like Ansor [13], BOLT [25], Chimera [5], and FlashAttention [30], [32] have ventured into the fusion of MBCI operators, they each confront their own search space constraints. Ansor's loop structure scheduling expands the search space but at the cost of efficiency [33]. BOLT facilitates the fusion of dual GEMM operators through template matching yet struggles with more complex modules like self-attention, which are not included in its fusion patterns. FlashAttention optimizes self-attention but requires extensive expert knowledge and lacks versatility for different workloads. Chimera attempts a comprehensive fusion strategy for compute-intensive operators yet is limited by a search space confined to nested loop permutations. The collective limitations of these frameworks highlight a critical gap in existing methodologies, underscoring the necessity for a more adaptive and encompassing search strategy to unlock the full potential of MBCI operator fusion.

b) Redundant Memory Access: Operator fusion aims to reduce memory accesses by storing intermediate tensors in shared memory. However, the fusion of multiple compute-intensive operators often leads to redundant memory access, as the coupling of computation blocks and loops may not always optimize memory usage. For example, loops specific to producer operators that also include consumer operators can cause unnecessary repeated accesses to the same memory region. Frameworks like Chimera attempt to optimize memory access by moving statements to the outermost relevant loops, a strategy mirrored in Ansor's `GetLastReduceIteratorInOutermostReduceTile` function. Despite these efforts, both frameworks miss opportunities to minimize access overhead in scenarios where loop extents are trivially one, highlighting a gap in fully leveraging memory access optimization.

c) Exploration and Tuning Cost: The search for the optimal schedule incurs significant time and resource costs across many compilers, such as TVM [14], Halide [34], and FlexTensor [35]. For instance, Ansor's extensive search space, navigated via an ML model like XGBoost, demands running statistics from online programs for model training, thereby increasing tuning overhead. While BOLT alleviates some of this burden through its template-based design, it still requires considerable template instantiation and program measurement, adding to the tuning effort. On the other hand, Chimera attempts to lower these costs using an analytical

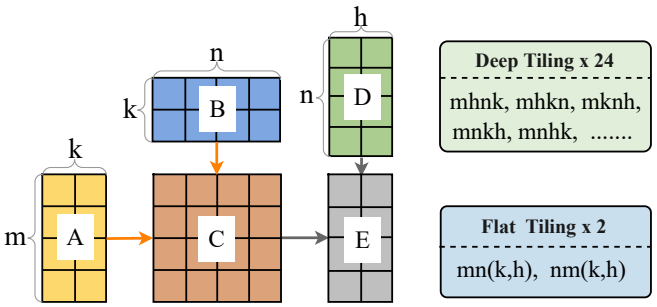


Fig. 3. The GEMM chain consists of two GEMM operators, with m, k, n, h representing the number of tiles in each dimension.

```
1  for i1, i2 in grid(m, h):
2    for i3 in range(n):
3      for i4 in range(k):
4        Load (tile A)
5        Load (tile B)
6        Compute (tile C)
7      Load (tile D)
8      Compute (tile E)
9      Store (tile E)        Opt.
10   Store (tile E)
```

(a) Memory node S_E moved from within loop n to loop h.

```
1  for i1 in range(m):
2    Load (tile A)           Opt.
3    for i2 in range(h):
4      for i3 in range(n):
5        Load (tile A)
6        Load (tile B)
7        Compute (tile C)
8      Load (tile D)
9      Compute (tile E)
10     Store (tile E)
```

(b) Memory node L_A shifted from within loop k to loop m ($k = 1$).

Fig. 4. The pseudo-code that illustrates the optimized tiling expression $mhnk$ for a fused GEMM chain, highlighting memory node optimization.

model focused on data transfer optimization. However, by neglecting the computational redundancy, it often arrives at sub-optimal scheduling decisions.

III. Search Space Generation and Optimization

In response to these identified challenges, MCFusor is designed with a comprehensive search space derived from tiling-based expressions. Through the utilization of DAG flow analysis, we efficiently minimize redundant memory access and filter out invalid search candidates, significantly streamlining the exploration process for enhanced efficiency.

A. Search Space Generation

In deep learning models, compute-intensive operators can be broken down into computation blocks surrounded by cross-tile loops, where each block iteratively accesses tiles of input tensors [36]–[38]. These loops are represented by a vector $\vec{l} = (l_1, l2, ...l_J)$, where J denotes the number of independent loops and l_j signifies the j-th loop with l_j representing its extent. Any candidate in the search space can be delineated by the structure of loops and the values of $\vec{l}$.

Initially, we utilize a straightforward tiling expression to delineate the structure of loops. The relationship between loops can be categorized into two types: Nested, where $l_j l_i$ signifies that the loop l_i is executed within the scope of l_j. Sequential, where (l_j, l_i) indicates that the loops l_i and l_j are executed sequentially within the same scope. For clarity, we exemplify with the GEMM chain scenario that comprises two compute-intensive operators. However, our analysis method naturally extends to scenarios with more compute-intensive operators. Additionally, we apply standard fusion optimizations for memory-intensive operators in line with previous work [7], [13], although such optimizations are beyond the scope of this paper.

As depicted in Fig. 3, the GEMM chain ($C = A \times B, E = C \times D$) is composed of four cross-tile loops: m, n, k, h, where m and n are shared loops for computation blocks C and E. Based on the loop relations outlined above, we categorize all tiling expressions into two categories:

- **Deep Tiling.** In this category, any two loops exhibit a nested relation, and the maximum depth of loops equals the number of loops. If there are x separate loops, then the number of permutations of loops is $x!$, such as $mnhk, mnkh, mkhn$, etc. For example, in the GEMM chain, this results in $4! = 24$ permutations in total.
- **Flat Tiling.** In this category, there exists loops with sequential relation, limiting the maximum depth of loops to be smaller than the number of loops. In the GEMM chain example, two flat tiling expressions exist: $mn(k, h)$ and $nm(k, h)$.

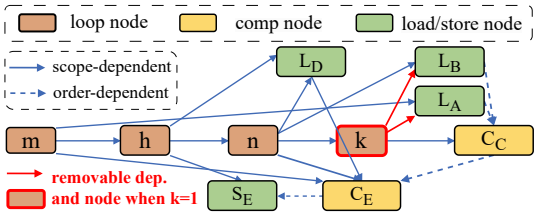
For the tile sizes $\vec{l}$, given that tensor cores require a minimum tile size of $16 \times 16 \times 16$, we consider all T_x that are multiples of 16 and smaller than the dimension size as viable options. Thus, the total number of candidates in our search space is determined by multiplying the number of possible tiling expressions with the viable tile size options.

Informed by our comprehensive analysis of tiling expressions and tile sizes, we are able to fully enumerate the candidates within the search space for fused kernels. Unlike Chimera, which focuses on deep tiling expressions to the exclusion of flat tiling, our approach ensures no potential configuration is overlooked. This distinction is crucial, as evidenced by the sub-optimal results of Chimera's strategy detailed in the experiments of Section VI-B.

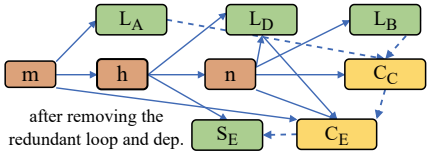
B. Memory Access Optimization

We enhance the basic tiling expression, addressing cross-tile loops, by adding details on computation and memory access. It introduces three primitive operations: $Load$ (L), $Compute$ (C), and $Store$ (S), denoted as suffixes attached to each operation, indicating the tensor it operates on. The position of $Compute$ primitive statements is determined by the rightmost related loops. By default, $Load$ and $Store$ primitive statements are associated with the corresponding $Compute$ statement. For instance, considering the GEMM chain depicted in Fig. 3, the related loops for computation blocks C and E are m, k, n and m, n, h, respectively. The basic expression of $mhnk$ can be expanded to $mhn(k(L_A, L_B, C_C), L_D, C_E, S_E)$, as illustrated in lines 1-9 of Fig. 4(a).

The volume of data movement for each tensor is calculated as the product of tile size and the trip counts of surrounding

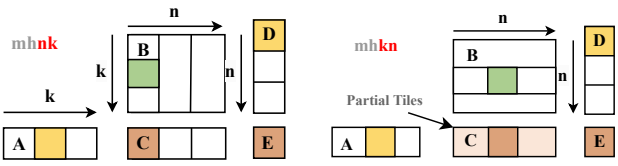


(a) The DAG of $mhnk$.



(b) The DAG of $mhnk, k = 1$.

Fig. 5. DAG flow illustrating the data processing steps for the tiling expression $mhnk$ in a fused GEMM chain.



(a) The sub-tiling expression is nk per thread block. (b) The sub-tiling expression is kn per thread block.

Fig. 6. Illustration of shared memory usage differences between tiling expressions $mhnk$ and $nhkn$. In (b), additional shared memory is needed to cache multiple tiles of C, whereas in (a), the memory footprint for a single tile can be efficiently reused.

loops. However, if the loop variables surrounding a certain operation are not utilized for accessing tensor tiles, it results in redundant operations. To mitigate this, memory access statements can be relocated to the rightmost related loop. For example, as depicted in Fig. 4(a), line 9 shows S_E within the m, h, k loops, but since k is not used to access tile E, moving the $Store$(Tile E) to the h loop scope, as in line 10, cuts memory accesses by a factor of k. This optimization, represented by the tiling expression $mh(n(k(L_A, L_B, C_C), L_D, C_E), S_E)$, reduces memory traffic. Although this method has been adopted by frameworks like Ansor and Chimera, it is often possible to further reduce redundancy, especially when the optimal tile size matches the dimension size, making some loop variables superfluous. In such scenarios, memory access efficiency is improved by adjusting the statements to outer loops, thus decreasing unnecessary trip counts.

To illustrate this scenario, all loops and primitive statements are organized into a DAG. The dependency from loop to primitive statements is scope-dependent, indicating that the statement must be executed within the loop's scope as the loop variables index the operands. The dependency between primitive statements is order-dependent, implying that one statement must be executed before another, but they need not be executed within the same loop's scope. In the example of $mh(n(k(L_A, L_B, C_C), L_D, C_E), S_E)$, the DAG representation (Fig. 5(a)) indicates that node L_A is dominated by loop node k, requiring it to be placed within the scope of loops m, h, n, k. Consequently, the matrix A is repeatedly loaded from global memory to shared memory hn times. However, when the extent of loop k decreases to 1, the loop variable becomes a constant value of 0, enabling the removal of the dead node k and its corresponding dependencies from the DAG, i.e., Fig. 5(b). As a result, the node L_A is primarily governed by loop m, which enables relocating its associated memory access statement into the m loop's scope (as shown in line 2 of Fig. 4(b)), thereby decreasing the access cost for L_A by a factor of hn.

With these optimizations, MCFuser significantly reduces redundant memory accesses for each candidate in the search space, substantially easing memory bandwidth demand.

C. Pruning

The size of the search space is determined by the number of tiling expressions multiplied by the number of candidate tile sizes. In the GEMM chain example with $M = N = 1024, K = H = 512$, this yields $(24 + 2) \times \lceil 1024/16 \rceil^2 \times \lceil 512/16 \rceil^2 = 109051904$ candidates, a substantial number. However, a significant portion of these candidates are either invalid or equivalent, leading to a waste of searching resources. To efficiently explore the search space and avoid meaningless trials, we propose several guidelines to prune the search space:

- **Rule 1, Deduplication**: We prioritize binding spatial loops with $blockIdx$ for each tiling expression, removing the loops bound to $blockIdx$ to derive the sub-tiling expression. Thus, candidates sharing a sub-tiling expression per thread block are deemed equivalent. For instance, both $mhnk$ and $mnkh$ yield the same sub-tiling expression nk for each thread block.
- **Rule 2, Prevent Overwhelming Intermediate Tensor's Shared Memory**: When the reduced loop is positioned outside the spatial loops, multiple tiles of partial results are cached in shared memory (as shown in Fig. 6), potentially overwhelming it.
- **Rule 3, Avoid Extra Padding**: Padding becomes necessary when the tile size does not evenly divide the dimension size. To avoid excessive padding, candidates requiring padding are discarded if the dimension size is a power of 2. For other cases, the padding ratio is kept below 0.05.
- **Rule 4, Shared Memory Limitation**: Candidates with tiles exceeding the shared memory limitation are considered illegal. We estimate the occupied shared memory as in equation (1), where Shm_{max} is the maximum supported shared memory for each block per specific hardware. We prune candidates that satisfy $Shm_{estm} > 1.2Shm_{max}$, where 1.2 accommodates estimation errors. As validated in experiments (Section VI-E1), our estimation accuracy exceeds 90%, resulting in approximately a 40% reduction in candidates.

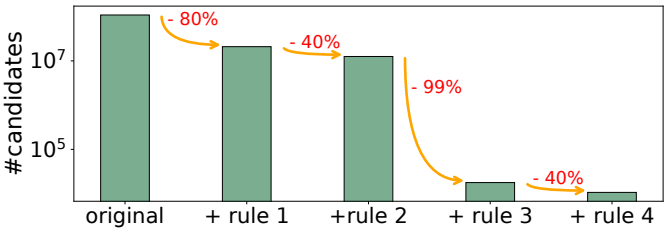


Fig. 7. The results of pruning search space candidates on an example of GEMM chain with $M = N = 1024, K = H = 512$.

$$Shm_{estm} = \sum_{\{Xi\}, Xi \in \mathbb{R}^{L_i \times L_j}} (T_{L_i} \times T_{L_j}) \quad (1)$$

Applying these rules to the GEMM chain example, we observe a drastic reduction in the search space size as illustrated in Fig. 7. Rule 1 reduces the number of tiling expressions from 26 to 5, and Rule 2 further reduces it to 3. Rule 3 discards 99% of impractical tile sizes, while Rule 4 eliminates 40% of infeasible tile sizes. Ultimately, the size of the search space is reduced from 10^8 to 10^4 after removing trivial candidates, facilitating the exploration of optimal candidates efficiently.

IV. Performance Model and Exploration

Previously, we crafted a comprehensive and efficient search space. We now turn our attention to its exploration, starting with an analytical performance model based on a detailed analysis of computation and memory access patterns. Further, we develop a heuristic search algorithm to automatically and efficiently pinpoint the optimal scheduling.

A. Analytical Performance Model

Auto tuners frequently grapple with the issue of protracted tuning times [13], [22]. To circumvent the repetitive and time-intensive performance testing, practitioners often resort to machine learning-based cost models to estimate program performance. Nonetheless, the extensive training and feature engineering required by these models can impose significant burdens on users. As a remedy, we introduce an analytical performance model that estimates the performance of generated tensor programs, detailed in equation (2). This model integrates three critical components: memory access overhead, computation overhead, and a slowdown factor, offering a streamlined approach to performance estimation.

Assuming a full schedule candidate consists of a set of computation blocks $\{C_{Xi}\}$, a set of memory access statements $\{L_{Xi}\} + \{S_{Xi}\}$, and a set of loops $(l_1, l_2, ..., l_J)$, where each compute or memory access statement St is surrounded by specific loops denoted as $Lp_set(St) = (l_{p1(St)}, l_{p2(St)}, ..., l_{pk(St)})$, with $pi(St)$ representing the i-th surrounding loop for statement St. The memory access overhead (equation (3)) for statement St_{Xi} is determined by the product of the tile size TS_{Xi} and the trip count $\prod_{l_j \in LP_set(St)} l_j$, divided by the memory bandwidth $\mathcal{W}$. Similarly, the computation overhead is estimated by the product of the flops of a tile Fp_{Xi} and the trip count, divided by the computation throughput $\mathcal{P}$, as in equation (4). Additionally, the number of thread blocks affects the parallelism of fused kernels. Since each thread block is dispatched on a SM, having more thread blocks helps exploit the parallel processing power provided by the GPU. Hence, we introduce a slowdown factor in equation (5), where $\mathcal{N}_{SM}$ denotes the number of SMs on the GPU and $\mathcal{N}_{block}$ denotes the number of thread blocks for the fused kernel. The slowdown factor α decreases with the value of $\mathcal{N}_{block}$ increasing until α approaches 1.

Incorporating an analytical performance model into MCFuser allows us to substantially diminish the necessity for runtime measurements, thereby significantly expediting the tuning process. Besides these, unlike approaches relying on cost models for performance estimation [13], [35], our method eliminates the requirement for model training, thereby reducing the tuning overhead for fused kernels.

$$t_{estm} = (t_{mem} + t_{comp}) \times \alpha \quad (2)$$

$$t_{mem} = \sum_{S_{Xi} \in \{L_{Xi}\} + \{S_{Xi}\}} \frac{TS_{Xi} \times \prod_{l_j \in LP_set(S)} l_j}{\mathcal{W}} \quad (3)$$

$$t_{comp} = \sum_{S_{Xi} \in \{C_{Xi}\}} \frac{Fp_{Xi} \times \prod_{l_j \in LP_set(S)} l_j}{\mathcal{P}} \quad (4)$$

$$\alpha = (\mathcal{N}_{block} + \mathcal{N}_{SM})/\mathcal{N}_{block} \quad (5)$$

B. Exploration

Despite pruning the search space, as discussed in Section III-C, a substantial number of candidates often remain. To streamline the selection process, we've refined the evolutionary search algorithm initially developed in Ansor with two key

Algorithm 1: Heuristic Search Algorithm

```
   parameters: N, n, ϵ
   input     : P, the input tensor program
   output    : cand_best, the best searched candidate
1  population = generateCandidates (P, N) ;
2  best_t = 1e9 ;
3  best_cand = Null ;
4  while True do
5      estimated_ts = estimatePerformance (population) ;
6      population = sort (population, key=estimated_ts) ;
7      topk_cands = topk (population, k=n, reverse=True) ;
8      topk_ts = measure (topk_cands) ;
9      top1_t, top1_cand = top1 (topk_ts, topk_cands) ;
10     if difference (top1_t, best_t) < ϵ then
11         best_cand = top1_cand; break;
12     end
13     if top1_t < best_t then
14         best_cand = top1_cand ;
15         best_t = top1_t ;
16     end
       /* generate next population                    */
17     population = mutate (population, weight=estimated_ts) ;
18 end
19 Return best_cand ;
```

enhancements to boost search efficiency. First, we've replaced Ansor's cost model with our analytical performance model, detailed in Section IV-A, eliminating the time-consuming model training phase and thereby reducing computational overhead. Second, unlike Ansor, which requires manually setting the number of tuning iterations, a process that demands considerable trial and error, we have implemented a convergence criterion that automatically terminates the search, further simplifying the optimization process.

Algorithm 1 delineates our methodology. Initially, we randomly select N candidates from the search space to establish the initial population (line 1). Utilizing our performance model from Section IV-A, we then estimate the execution time for each candidate within this population (line 5), proceeding to isolate the top n candidates (significantly fewer than N) for detailed measurement. Should the gap between the $top1_t$ and the foremost measured result within this group fall below ϵ, signifying convergence, the search process halts (lines 10-12). If not, we refresh the best result (lines 13-16) and craft the subsequent population (line 17), deriving it through mutation from its predecessor. Assuming the preceding population is $Cand_i$, with estimated execution times denoted by $1/et_i$ for $i = 1, 2, ..., N$, each candidate $Cand_i$ is assigned a weight of $1/et_i$. We then randomly draw N candidates from $Cand_i$, weighted by these values. For each selected candidate, one loop is chosen to mutate the tile size, generating a new candidate for the next population.

Through search space pruning and the integration of a heuristic search algorithm with an analytical performance model, MCFuser effectively automates and optimizes the search for the optimal fused kernel implementation.

V. Implementation

MCFuser is built on top of TVM [14] and Triton [39].

A. Intra-block optimization and Code Generation

Triton [39], an open-source language and compiler, excels in expressing and compiling tiled neural network computations into efficient machine code. With its user-friendly interfaces, Triton enables developers to adeptly design high-performance compute kernels at the tile level, managing intra-tile optimizations autonomously.

In MCFuser, we concentrate on leveraging Triton for inter-tile optimization, utilizing its suite of intra-tile optimizations. These include automatic coalescing, thread swizzling, automatic vectorization, tensor core-aware instruction selection, and efficient shared memory allocation and synchronization. This approach simplifies the optimization process and ensures the generated code is highly efficient.

Moreover, we harness Triton's capabilities to produce optimized PTX (Parallel Thread Execution) code [40], which is then seamlessly integrated with TVM, as detailed in the following subsection. This integration leverages the strengths of both Triton and TVM, creating an effective and adaptable optimization pipeline for tensor programs.

TABLE II
The Configuration of Batch GEMM chains.

Name	batch	M	N	K	H
G1/G2/G3	1	512	256	64	64/128/256
G4/G5/G6	1	512	512	256/512/1024	256
G7/G8/G9	1	512/1024/2048	512	128	128
G10/G11/G12	1/4/8	1024	1024	128	128

TABLE III
The Configurations of Self Attention Modules.

Name	#heads	M	N	K	H	Network
S1	8	512	512	64	64	Bert-Small
S2	12	512	512	64	64	Bert-Base
S3	16	512	512	64	64	Bert-Large
S4	12	256	256	64	64	ViT-Base
S5	16	256	256	64	64	ViT-Large
S6	16	256	256	80	80	ViT-Huge
S7/S8/S9	1	512/768/1024	256/384/512	64	64	MLP-Mixer

B. Framework and Front-end

To streamline our efforts and avoid redundant work, we leverage the existing compiler stack provided by TVM to the fullest extent possible. This includes utilizing the front-end languages Relay IR and TIR to express models, perform graph-level transformations, and handle runtime operators.

When processing a MBCI operator defined in Relay IR, we initially translate it into both a tiling expression and a TIR representation. These two representations are mutually convertible. The transition from a tiling expression to a TIR module is facilitated using `tvm.tir.Schedule` primitives such as $tile$, $split$, $reorder$, and $bind$. To further streamline this process, we have crafted a TIR AST visitor that extracts the tiling expression directly from a TIR module. An IR translator has also been developed to morph a TIR module into a TritonIR module. Finally, the PTX code produced by Triton is converted into the TVM runtime library using the `runtime.module.loadfile_ptx` interface and encapsulated within an `OperatorModule` for evaluation.

When presented with a deep learning model originating from frameworks like PyTorch [26], TensorFlow [41], or ONNX [42], we initially convert it into the Relay module. Subsequently, we employ a partitioner to segment the model into MBCI sub-graphs and other components. The MBCI parts are then optimized using MCFuser, and the optimized PTX code is incorporated into the TVM runtime module. For other operators, we either continue optimization with Ansor or Relay, a decision determined flexibly based on experiments outlined in Section VI-C. Finally, all runtime libraries for the operators are integrated into the executable `GraphExecutorFactoryModule`.

By leveraging the capabilities of TVM's compiler stack, we ensure efficient handling of both MBCI operators and other components of deep learning models, facilitating seamless integration and optimization throughout the compilation process.

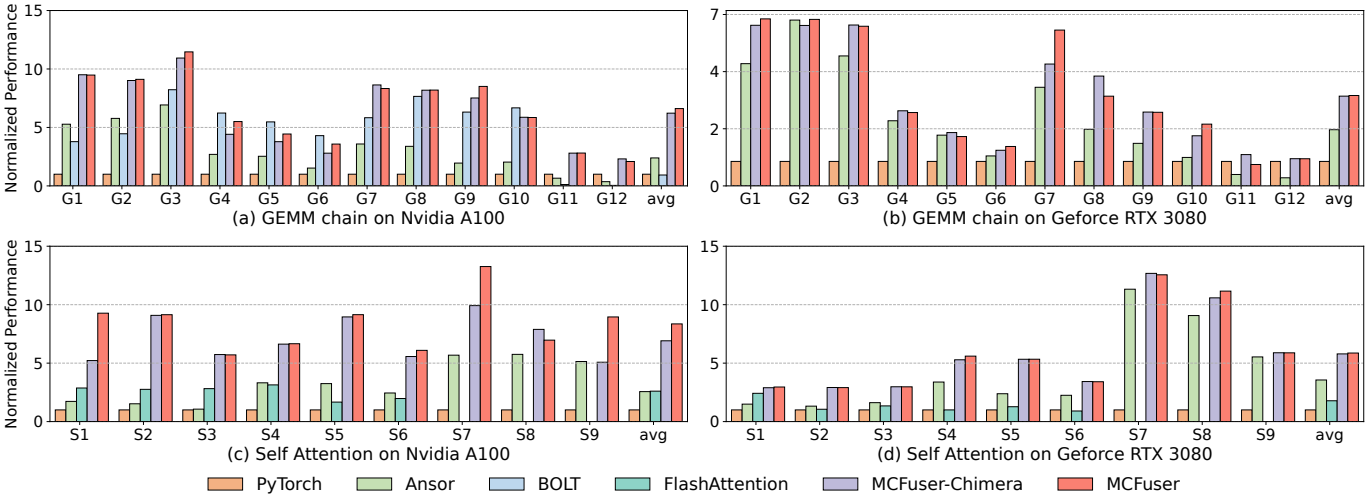


Fig. 8. The relative performance of fusing batch GEMM chains and self-attention modules on GPUs.

VI. Evaluation

A. Experimental Setup

Platforms. We evaluate MCFuser on two servers. The first is equipped with one NVIDIA A100-PCIE with 40GB and 72-core Intel(R) Xeon(R) Gold 6240C CPUs clocked at 2.60GHz. The second server features one NVIDIA GeForce RTX 3080 GPU and a 20-core Intel(R) Core(TM) i9-10900X CPU running at 3.70GHz. For simplicity, we refer to these servers as A100 and RTX3080 in the experiments. The major compilers and libraries utilized by MCFuser and other baselines include TVM 0.14, Triton 2.1, CUDA 11.7, FlashAttention[1], Transformers 4.16[2], and PyTorch 2.1.

Workloads. We assess both sub-graph fusion performance and end-to-end network performance. The sub-graphs we analyze comprise various GEMM operator chains, as well as self-attention modules from Bert [43], Vit [44], and MLP-Mixer [45]. Additionally, we evaluate the performance of end-to-end networks on Bert models.

Comparisons. Our baselines encompass both hand-tuned libraries and cutting-edge compilers. For libraries, we compare against PyTorch [26] (utilizing CuBlas [8] and CuDNN [9]) and FlashAttention [30] (optimized for self-attention modules). Regarding compilers, we compare against leading machine learning compilers, including Relay [46], Ansor [13], BOLT [25] (built upon TVM [14] and cutlass [10]). For Ansor, we conduct 1000 tuning trials for each subgraph. To ensure a rigorous assessment of our search space generation effectiveness against the closed-source Chimera [5], we implement MCFuser-Chimera. This adaptation integrates Chimera's search space into our framework, allowing for a direct and controlled comparison.

[1]https://github.com/HazyResearch/flash-attention.git@57ee618
[2]https://github.com/huggingface/transformers

B. Subgraph Performance

1) Batch GEMM Chain: The configurations for the batch GEMM chains are detailed in Table II. Here, the dimension $(batch, M, K) \times (batch, K, N)$ specifies the size for the first batch GEMM operator, while $(batch, M, N) \times (batch, N, H)$ delineates the size of the subsequent batch GEMM operator.

Figures 8(a) and (b) depict the performance of subgraphs normalized to PyTorch. For the A100 and RTX 3080 GPUs, the average speedup is 6.6× and 3.7× compared to PyTorch, 2.7× and 1.6× compared to Ansor, and 1.06× and 1.07× compared to MCFuser-Chimera. However, except for the cases where MCFuser and MCFuser-Chimera produce identical fused kernels, e.g. G1, G8 in Fig. 8(a), MCFuser achieves 1.17× and 1.13× speedups compared to MCFuser-Chimera on A100 and RTX 3080 GPUs, respectively, underscoring the efficiency of our search space generation approach. Additionally, MCFuser delivers a 7.1× speedup over BOLT on A100. Note that BOLT does not support GPUs with *sm86* compute capability, including RTX 3080.

The speedup compared to PyTorch is attributed to fusing the memory-bound batch GEMMs and reducing off-chip memory access. Ansor adopts declarative loop-oriented scheduling but suffers from inefficient schedule space and lengthy tuning time [33]. Additionally, we achieve up to 5.9× and speedup compared to Ansor in the cases where Ansor fails to fuse the GEMM chains (e.g., G12 in Fig. 8(a)); this is because it concentrates more on the optimization of general operators, leading to sub-optimal performance of fusing MBCI operators. BOLT leverages Cutlass, the state-of-the-art open-source DNN template library for GPUs, to generate fused kernels. However, its flexibility is limited as the templates are manually developed by experts. Moreover, BOLT performs worse in some extreme cases of large tensor size, such as G11 and G12. In comparison with MCFuser-Chimera, MCFuser encompasses larger search spaces, as introduced in Section III-A, leading to better performance in various cases (e.g., G3, G4, G5 as

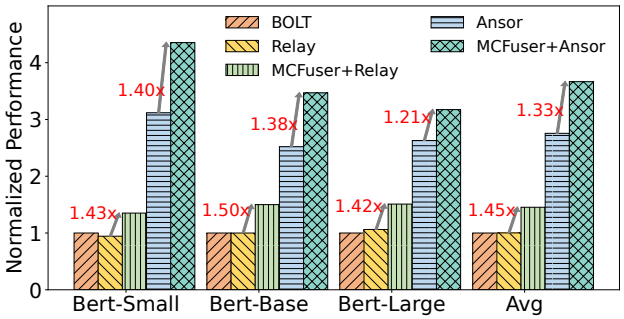


Fig. 9. The end-to-end model evaluation on A100 GPU.

depicted in Fig. 8(a).

2) Self-Attention Module: The input configurations for the self-attention modules, detailed in Table III, employ the same dimensional parameters (M, N, K, and H) as used in the GEMM Chains. Unlike GEMM chains, the self-attention module includes a broader range of operators, such as softmax, which is further divisible into a series of smaller operators.

Figures 8(c) and 8(d) showcase the performance of self-attention modules normalized to PyTorch. For the A100 and RTX 3080 GPUs, the average speedup is 8.1× and 5.8× compared to PyTorch, 2.8× and 1.45× compared to Ansor, 3.0× and 3.3× compared to FlashAttention, and 1.1× and 1.01× compared to MCFuser-Chimera, respectively. While FlashAttention employs a clever design for fusing self-attention modules, its implementation suffers from inflexibility. Specifically, a rigid constraint of $K = H$ prevents it from fusing specific modules where K and H differ. Additionally, FlashAttention only considers splitting the M and N dimensions into tiles, neglecting K and H, which likely results in sub-optimal performance for many cases. BOLT lacks the ability to fuse self-attention modules due to its constrained fusion patterns.

The results underscore MCFuser's capability in achieving high-performance fusion for memory-bound compute-intensive operators, validating its efficiency and effectiveness.

C. End-to-End Performance

For the end-to-end network performance evaluation, we utilize *Bert-Small*, *Bert-Base*, and *Bert-Large* as workloads, setting the sequence length to 512. The key configurations of these models are listed in Table III. We employ BOLTs as the baseline. Despite BOLT's limitations in handling self-attention modules, it incorporates optimizations such as epilogue fusions (e.g., GEMM+bias+ReLU). Additionally, we compare *MCFuser* with Relay and Ansor. Since *MCFuser* primarily focuses on fusing memory-bound compute-intensive operators, we integrate it with Relay and Ansor to leverage their broader operator coverage. The variants of *MCFuser* for end-to-end networks are denoted as *MCFuser+Relay* and *MCFuser+Ansor*, respectively.

Our comprehensive evaluation, performed on an A100 GPU, is depicted in Fig. 9. It reveals that Relay, while a robust framework, delivers suboptimal performance due to its dependence on pre-defined templates without subsequent fine-tuning. Conversely, BOLT faces limitations from its pattern table, which restricts it to fusing a narrow set of operators. This approach largely leaves BOLT dependent on Relay's implementations, resulting in only slight improvements.

In contrast, *MCFuser+Relay* significantly outperforms Relay alone, achieving an impressive average speedup of 1.45×. This performance is further elevated when integrated with *Ansor*, with *MCFuser+Ansor* vastly outperforming *BOLT* by securing an average speedup of 3.66×. Remarkably, even against Ansor, which is celebrated for its highly optimized kernels, *MCFuser+Relay* demonstrates superior performance, boasting an average speedup of 1.33×. This analysis underscores the effectiveness of *MCFuser* in elevating the computational efficiency across varying platforms.

TABLE IV
TUNING TIMES FOR SUB-GRAPH MODULES AND END-TO-END MODELS. "MCFUSER+RELAY" AND "MCFUSER+ANSOR" COLUMNS INDICATE THE RESPECTIVE SPEEDUPS RELATIVE TO *BOLT* AND *Ansor*.

Sub Graph					
	BOLT	Ansor	MCFuser-Chimera	MCFuser	Speedup to BOLT/Ansor
GEMM Chain	88s	4895s	29s	35s	**2.5× / 139×**
Self Attention	-	2897s	32s	39s	**- / 74×**
End-to-End					
methods	Relay	BOLT	MCFuser+Relay	Ansor	MCFuser+Ansor
Bert-Small	30s	94s	81s (**1.16×**)	4.03h	2.78h (**1.45×**)
Bert-Base	149s	227s	202s (**1.12×**)	3.94h	2.74h (**1.43×**)
Bert-Large	186s	383s	243s (**1.57×**)	4.06h	2.98h (**1.36×**)

D. Tuning time

Table IV provides detailed tuning time for both sub-graphs and end-to-end networks on the A100 GPU. When tuning sub-graph modules, MCFuser significantly outperforms BOLT (2.5× faster on average) and Ansor (139× faster for GEMM chains and 74× faster for self-attention modules on average). This speedup is attributed to MCFuser's efficient search space and analytical performance model, which eliminates the time-consuming hardware profiling and model training required by Ansor, as well as the extensive template instantiation and massive program measurements in BOLT.

In the context of end-to-end network tuning, our first variant, *MCFuser+Relay*, significantly outperforms both Relay and BOLT, as depicted in Fig. 9. This enhancement is realized with a minimal time cost increase over Relay (under one minute) and achieves a tuning time reduction ranging from 1.12× to 1.57× compared to BOLT. Moreover, *MCFuser+Ansor* attains a 1.41× average speedup in tuning time over Ansor alone. Notably, the bulk of the tuning time for *MCFuser+Ansor* is consumed by operators processed by Ansor. To further reduce the tuning time, our future work will aim to expand MCFuser's framework to include a broader array of operators.

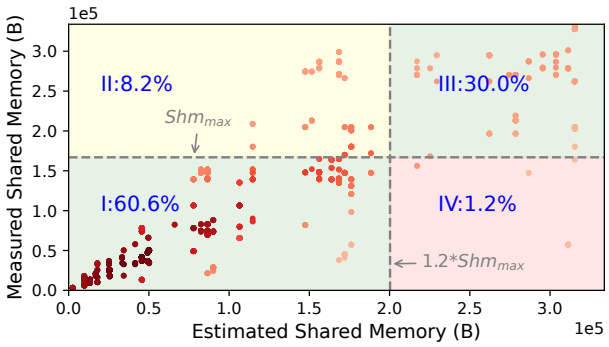


Fig. 10. Model-predicted vs. actual shared memory usage on the NVIDIA A100 GPU, with Shm_{max} denoting the maximum usable shared memory per thread block, factoring in special optimizations. The color shading of data points indicates data density.

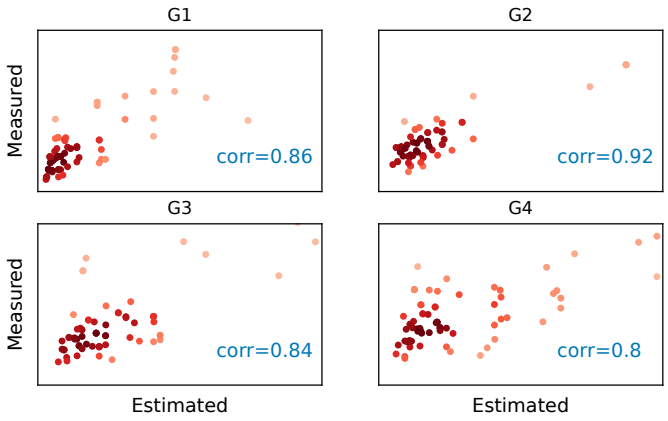


Fig. 11. The model predicted performance versus actual measurement on NVIDIA A100 GPU. The closer to the zero point, the better the performance. The shade of the points' color indicates the density of the data.

E. Effectiveness of the System Design

Our system design employs Equation (1) to estimate shared memory usage, effectively pruning infeasible scheduling candidates. Furthermore, an analytical performance model is utilized to forecast the performance of candidates, facilitating their selection. To validate the efficacy of these approaches, we undertake the following experimental procedures.

1) Shared Memory Estimation: Figure 10 displays the relationship between estimated shared memory usage per thread block and actual measurements performed by the NVPTX backend in LLVM, as applied to scheduled candidates from various experiments detailed in Section VI-B. This figure is segmented into four quadrants, distinguished by the y-axis at Shm_{max}, the maximal shared memory allotment per thread block, beyond which candidates are considered unexecutable on GPUs. The x-axis bifurcation at $1.2\times Shm_{max}$, an empirically determined threshold, aims to adequately account for estimation inaccuracies.

Notably, over 90% of data points reside within quadrants I and III, affirming the precision of our estimation methodology in aptly identifying and pruning unviable schedules, thus curtailing the search space by 40%. While 8.2% of candidates (quadrant II) that should have been pruned remain, their influence on the final program performance is minimal, as they are subsequently eliminated during PTX code lowering. A mere 1.2% of candidates are not correctly dropped despite nearing the Shm_{max} limit, yet upon closer examination, these cases are inconsequential due to their considerably inferior performance compared to the optimal selection.

2) Analytical Performance Model: To assess our analytical performance model's accuracy, we examined the correlation between the estimated and actual performance of scheduled candidates for the first four cases listed in Table II. As shown in Fig.11, the correlation coefficients 0.86, 0.92, 0.84, and 0.8 underscore the reliability of our analytical performance model. As detailed in Algorithm1, the model identifies the top n candidates (where n is empirically set to 8) for further evaluation, enhancing our chances of identifying the optimal solution. Furthermore, the concentration of data points near the origin in Fig. 11 highlights the effectiveness and expressiveness of our search space generation approach.

VII. Related Work

Various hand-tuned libraries [8], [9], [47], [48] and code generation compilers [6], [13]–[20], [22], [46], [49] have been developed to enhance the performance of Deep Learning models. Among these efforts, operator fusion stands out as a prevalent optimization technique aimed at reducing kernel launch overhead and memory accesses. Depending on the type of operator and optimization method, we classify operator fusion into three categories: hand-tuned or library-based, memory-intensive, and compute-intensive.

Hand-tuned or library-based Wang et al. [50] investigate fusing convolutional layers in CNNs, while Liang et al. [51] focus on maximizing hardware utilization through spatial-temporal kernel fusion. Astra [16] is notable for fusing GEMM operators in RNNs, albeit lacking low-level code generation and relying on hand-tuned libraries. Ashari et al. [52] implement fused kernels for specific machine learning patterns, and FlashAttention [30], [32] provides hand-tuned optimizations for self-attention modules. While these approaches achieve high performance, their reliance on hand-tuned kernels or manual optimization limits their flexibility and adaptability to new operators. In contrast, MCFuser's integration within a compiler framework offers a more generalizable solution for optimizing diverse operators.

Memory-Intensive Operators Numerous compilers [14], [15], [20], [21], [53]–[55] employ fusion optimizations for memory-intensive operators. However, these techniques often struggle to address complex dependencies, resulting in incomplete fusion outcomes. Works such as Fusion Stitching [29], and AStitch [7] attempt to expand fusion scope by utilizing shared and global memory for intermediate storage. However, by treating compute-intensive operators as fusion boundaries, they overlook valuable optimization opportunities that fusing these operators would provide. In comparison, MCFuser leverages the identification of memory-bounded compute-intensive

operators to push the boundaries of operator fusion, enabling more creative and efficient fusion designs.

Compute-Intensive Operators Optimization of compute-intensive operators has been explored by several previous works [13]–[15], [20], [53], [54], [56], [57]. However, most of these works are limited to scenarios that fuse one compute-intensive operator with non-compute-intensive ones. Frameworks like NeoFlow [58] avoid fusing multiple compute-intensive operators entirely due to code generation constraints. DNNFusion [6], designed for mobile devices (e.g., ARM CPU and GPU), fails to fuse compute-intensive operators because its fusion algorithm always considers fusing compute-intensive operators as non-beneficial. BOLT [25] relies on the Cutlass [10] template library to generate code for fused GEMM, and operator types are strictly constrained by predefined pattern tables (e.g., BOLT can't recognize einsum operators). Chimera [5] focuses on minimizing data movement in block execution, neglecting the impact of redundant computation, but its search space is limited by the block execution order.

In contrast, we present an exhaustive fusion strategy that generates a comprehensive search space, complemented by an analytical model that efficiently captures both memory and computation overheads for superior design choices.

VIII. Conclusion

In this work, we have unveiled fusion opportunities for a novel class of operators known as MBCI operators. By developing a tiling expression that precisely defines the search space, we have facilitated the efficient fusion of these operators, thereby enhancing the generation of high-performance tensor programs. Through DAG analysis and memory access optimization, coupled with strategic pruning guidelines and the integration of an analytical performance model with a heuristic search approach, we streamlined the tuning process. Validated on NVIDIA A100 and RTX 3080 GPUs, our methods demonstrated the capability of MCFuser to significantly elevate performance, achieving up to a 5.9× speedup over Ansor, and surpassing other methods. Importantly, MCFuser also dramatically reduces tuning times by more than 70-fold, highlighting its potential to profoundly impact machine learning machine learning compilation through intelligent optimization and integration strategies.

Acknowledgment

This work was supported by the National Key Research and Development Program of China (2023YFE0205700), National Natural Science Foundation of China (62302348, 62341410), Fundamental Research Funds for the Central Universities (2042023kf0132), General Program of Hubei Provincial Natural Science Foundation of China (2023AFB831), Special Fund of Hubei Luojia Laboratory (220100016) and the Science and Technology Development Fund (FDCT) Macau SAR (File no. 0078/2023/AMJ). The numerical calculations in this paper have been done on the supercomputing system in the Supercomputing Center of Wuhan University.

References

[1] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, "Antman: Dynamic scaling on gpu clusters for deep learning," in *Proc. of USENIX OSDI*, 2020, pp. 533–548.

[2] X. Jia, L. Jiang, A. Wang, W. Xiao, Z. Shi, J. Zhang, X. Li, L. Chen, Y. Li, Z. Zheng *et al.*, "Whale: Efficient giant model training over heterogeneous gpus," in *USENIX Annual Technical Conference*, 2022, pp. 673–688.

[3] Z. Zhang, Y. Xia, H. Wang, D. Yang, C. Hu, X. Zhou, and D. Cheng, "Mpmoe: Memory efficient moe for pre-trained models with adaptive pipeline parallelism," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 843–856, 2024.

[4] Y. Xia, Z. Zhang, H. Wang, D. Yang, X. Zhou, and D. Cheng, "Redundancy-free high-performance dynamic GNN training with hierarchical pipeline parallelism," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing, HPDC 2023, Orlando, FL, USA, June 16-23, 2023*, A. R. Butt, N. Mi, and K. Chard, Eds. ACM, 2023, pp. 17–30.

[5] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, "Chimera: An analytical optimizing framework for effective compute-intensive operators fusion," in *IEEE International Symposium on High-Performance Computer Architecture, HPCA*, 2023, pp. 1113–1126.

[6] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021, pp. 883–898.

[7] Z. Zheng, X. Yang, P. Zhao, G. Long, K. Zhu, F. Zhu, W. Zhao, X. Liu, J. Yang, J. Zhai, S. L. Song, and W. Lin, "Astitch: enabling a new multi-dimensional optimization space for memory-intensive ML training and inference on modern SIMT architectures," in *ASPLOS '22*, 2022, pp. 359–373.

[8] "Nvidia cublas." [Online]. Available: https://developer.nvidia.com/cublas

[9] "Nvidia cudnn." [Online]. Available: https://developer.nvidia.com/cudnn

[10] "Nvidia cutlass." [Online]. Available: https://developer.nvidia.com/cutlass

[11] "Nvidia tensorrt." [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/ developer-guide/index.html

[12] Z. Zhang, D. Yang, Y. Xia, L. Ding, D. Tao, X. Zhou, and D. Cheng, "Mpipemoe: Memory efficient moe for pre-trained models with adaptive pipeline parallelism," in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, pp. 167–177.

[13] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI*, 2020, pp. 863–879.

[14] T. Chen, L. Zheng, E. Yan, Z. Jiang, T. Moreau, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Learning to optimize tensor programs," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[15] R. Baghdadi, J. Ray, M. B. Romdhane, E. D. Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. P. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *IEEE/ACM International Symposium on Code Generation and Optimization, CGO*, 2019.

[16] M. Sivathanu, T. Chugh, S. S. Singapuram, and L. Zhou, "Astra: Exploiting predictability to optimize deep learning," in *ASPLOS*, 2019, pp. 909–923.

[17] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, and A. Aiken, "TASO: optimizing deep learning computation with automatic generation of graph substitutions," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP*, 2019.

[18] M. Wahib and N. Maruyama, "Scalable kernel fusion for memory-bound GPU applications," in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*. IEEE Computer Society, 2014, pp. 191–202.

[19] J. Zhao, B. Li, W. Nie, Z. Geng, R. Zhang, X. Gao, B. Cheng, C. Wu, Y. Cheng, Z. Li, P. Di, K. Zhang, and X. Jin, "AKG: automatic kernel generation for neural processing units using polyhedral transformations," in *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2021.

[20] N. Vasilache, O. Zinenko, T. Theodoridis, P. Goyal, Z. DeVito, W. S. Moses, S. Verdoolaege, A. Adams, and A. Cohen, "Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions," *CoRR*, vol. abs/1802.04730, 2018.

[21] "Tensorflow xla." [Online]. Available: https://www.tensorflow.org/xla

[22] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *OSDI*, 2018, pp. 578–594.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[24] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[25] J. Xing, L. Wang, S. Zhang, J. Chen, A. Chen, and Y. Zhu, "Bolt: Bridging the gap between auto-tuners and hardware-native performance," in *Proceedings of Machine Learning and Systems*, 2022.

[26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

[27] Y. Zhai, C. Jiang, L. Wang, X. Jia, S. Zhang, Z. Chen, X. Liu, and Y. Zhu, "Bytetransformer: A high-performance transformer boosted for variable-length inputs," in *IEEE International Parallel and Distributed Processing Symposium, IPDPS*, 2023, pp. 344–355.

[28] W. Hulin, D. Yang, Y. Xia, Z. Zhang, Q. Wang, J. Fan, X. Zhou, and D. Cheng, "Raptor-t: A fused and memory-efficient sparse transformer for long and variable-length sequences." *IEEE Transactions on Computers*, vol. 73, pp. 1852–1865, 2024.

[29] Z. Zheng, P. Zhao, G. Long, F. Zhu, K. Zhu, W. Zhao, L. Diao, J. Yang, and W. Lin, "Fusionstitching: Boosting memory intensive computations for deep learning workloads," *CoRR*, vol. abs/2009.10924, 2020.

[30] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, 2022.

[31] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.

[32] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *CoRR*, vol. abs/2307.08691, 2023.

[33] Y. Ding, C. H. Yu, B. Zheng, Y. Liu, Y. Wang, and G. Pekhimenko, "Hidet: Task-mapping programming paradigm for deep learning tensor programs," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS*, 2023, pp. 370–384.

[34] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. P. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *SIGPLAN Conference on Programming Language Design and Implementation, PLDI*. ACM, 2013, pp. 519–530.

[35] S. Zheng, Y. Liang, S. Wang, R. Chen, and K. Sheng, "Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system," in *Architectural Support for Programming Languages and Operating Systems, ASPLOS*. ACM, 2020, pp. 859–873.

[36] S. Zheng, R. Chen, A. Wei, Y. Jin, Q. Han, L. Lu, B. Wu, X. Li, S. Yan, and Y. Liang, "AMOS: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction," in *ISCA '22: The 49th Annual International Symposium on Computer Architecture*. ACM, 2022, pp. 874–887.

[37] S. Zheng, X. Zhang, L. Liu, S. Wei, and S. Yin, "Atomic dataflow based graph-level workload orchestration for scalable DNN accelerators," in *International Symposium on High-Performance Computer Architecture, HPCA*. IEEE, 2022, pp. 475–489.

[38] H. Zhu, R. Wu, Y. Diao, S. Ke, H. Li, C. Zhang, J. Xue, L. Ma, Y. Xia, W. Cui, F. Yang, M. Yang, L. Zhou, A. Cidon, and G. Pekhimenko, "ROLLER: fast and efficient tensor compilation for deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI*, 2022, pp. 233–248.

[39] P. Tillet, H. Kung, and D. D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.

[40] "Nvidia, parallel thread execution isa." [Online]. Available: http://docs.nvidia.com/cuda/parallel-thread-execution/index.html

[41] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. A. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI*, 2016, pp. 265–283.

[42] "Onnx." [Online]. Available: https://onnx.ai/

[43] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[44] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *9th International Conference on Learning Representations, ICLR*, 2021.

[45] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy, "Mlp-mixer: An all-mlp architecture for vision," in *Advances in Neural Information Processing Systems*, 2021, pp. 24 261–24 272.

[46] J. Roesch, S. Lyubomirsky, M. Kirisame, J. Pollock, L. Weber, Z. Jiang, T. Chen, T. Moreau, and Z. Tatlock, "Relay: A high-level IR for deep learning," *CoRR*, vol. abs/1904.08368, 2019.

[47] "Intel oneapi deep neural network library." [Online]. Available: https://github.com/oneapi-src/oneDNN

[48] "Intel oneapi math kernel library." [Online]. Available: https://software.intel.com/content/ www/us/en/develop/tools/oneapi/-components/onemkl.html

[49] L. Ma, Z. Xie, Z. Yang, J. Xue, Y. Miao, W. Cui, W. Hu, F. Yang, L. Zhang, and L. Zhou, "Rammer: Enabling holistic deep learning compiler optimizations with rtasks," in *OSDI*, 2020.

[50] X. Wang, G. Li, X. Dong, J. Li, L. Liu, and X. Feng, "Accelerating deep learning inference with cross-layer data reuse on gpus," in *Euro-Par 2020: Parallel Processing - 26th International Conference on Parallel and Distributed Computing*, ser. Lecture Notes in Computer Science, vol. 12247, 2020, pp. 219–233.

[51] Y. Liang, H. P. Huynh, K. Rupnow, R. S. M. Goh, and D. Chen, "Efficient GPU spatial-temporal multitasking," *IEEE Trans. Parallel Distributed Syst.*, vol. 26, no. 3, pp. 748–760, 2015.

[52] A. Ashari, S. Tatikonda, M. Boehm, B. Reinwald, K. Campbell, J. Keenleyside, and P. Sadayappan, "On optimizing machine learning workloads via kernel fusion," in *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP*, 2015, pp. 173–182.

[53] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand, and J. Ragan-Kelley, "Learning to optimize halide with tree search and random programs," *ACM Trans. Graph.*, vol. 38, no. 4, pp. 121:1–121:12, 2019.

[54] H. Shen, J. Roesch, Z. Chen, W. Chen, Y. Wu, M. Li, V. Sharma, Z. Tatlock, and Y. Wang, "Nimble: Efficiently compiling dynamic neural networks for model inference," in *Proceedings of Machine Learning and Systems*, 2021.

[55] K. Zhu, W. Zhao, Z. Zheng, T. Guo, P. Zhao, J. Bai, J. Yang, X. Liu, L. Diao, and W. Lin, "DISC: A dynamic shape compiler for machine learning workloads," in *EuroMLSys@EuroSys 2021, Proceedings of the 1st Workshop on Machine Learning and Systems Virtual Event*, E. Yoneki and P. Patras, Eds. ACM, 2021, pp. 89–95.

[56] M. Cowan, T. Moreau, T. Chen, J. Bornholt, and L. Ceze, "Automatic generation of high-performance quantized machine learning kernels," in *CGO '20: 18th ACM/IEEE International Symposium on Code Generation and Optimization*, 2020, pp. 305–316.

[57] J. Kim, A. Sukumaran-Rajam, V. Thumma, S. Krishnamoorthy, A. Panyala, L. Pouchet, A. Rountev, and P. Sadayappan, "A code generator for high-performance tensor contractions on gpus," in *IEEE/ACM International Symposium on Code Generation and Optimization,CGO*, 2019, pp. 85–95.

[58] S. Zheng, R. Chen, Y. Jin, A. Wei, B. Wu, X. Li, S. Yan, and Y. Liang, "Neoflow: A flexible framework for enabling efficient compilation for high performance DNN training," *IEEE Trans. Parallel Distributed Syst.*, vol. 33, no. 11, pp. 3220–3232, 2022.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

In summary, our contributions are as follows:

- C_1 We support the fusion of MBCI (memory-bound compute-intensive) operators. First, we establish the complete search space based on comprehensive tiling expressions and eliminate redundant memory access based on DAG analysis. Then, we automate the fusion process by employing a heuristic search algorithm.
- C_2 MCFuser accelerates the execution of end-to-end deep learning models. We integrate MCFuser into the TVM framework and employ a partitioner to segment the model into MBCI sub-graphs and other components. MCFuser deals with the MBCI sub-graphs, and the other components are handled by Relay or Ansor. We denote two variants as MCFuser+Relay and MCFuser+Ansor, respectively.
- C_3 To streamline tuning costs and improve deployment efficiency, we propose efficient guidelines for pruning the search space and integrate an analytical performance model into a heuristic search method, enabling the identification of optimal schedules automatically and efficiently. Finally, MCFuser yields a substantial reduction in tuning time.

B. Computational Artifacts

A_1 https://doi.org/10.5281/zenodo.10971908

Artifact ID	Sub ID	Contributions Supported	Related Paper Elements
A_1	$A_{1.1}$	C_1	Figure 8(a)(b)(c)(d)
	$A_{1.2}$	C_2	Figure 9
	$A_{1.3}$	C_3	Tables IV

II. Artifact Identification

A. Computational Artifact A_1

For convenience, we archive all the computational artifacts under a single Zenodo link. These artifacts are highly correlated and share the same artifact setup. Additionally, A_1 not only contains all the computational artifacts but also the source code of this paper. For clarity, we distinguish the specific artifacts within A_1 as $A_{1.1}$, $A_{1.2}$, and $A_{1.3}$, as outlined in the above table.

Relation To Contributions

$A_{1.1}$**:** We assess the sub-graph fusion performance, including various GEMM chains and self-attention modules from Bert, Vit, and MLP-Mixer. To verify the effectiveness of the fusion of MBCI operators, C_1, We compare against hand-tuned libraries and cutting-edge compilers, including two libraries, PyTorch and FlashAttention, and the state-of-the-art machine learning compilers, Ansor and BOLT.

$A_{1.2}$: We verify MCFuser's capability of deploying end-to-end deep learning models (C_2). We conduct experiments on three models, including Bert-Small, Bert-Base, and Bert-Large, and compare MCFuser against state-of-the-art machine learning compilers, including BOLT, Relay, and Ansor.

$A_{1.3}$**:** We validate the tuning time of experiments in $A_{1.1}$ and $A_{1.2}$, demonstrating that MCFuser can save a substantial amount of tuning time and improve the deployment efficiency (C_3).

Expected Results

$A_{1.1}$**:** The fused kernels generated by MCFuser outperforms Pytorch, FlashAttention, BOLT, and Ansor in most workloads. It is worth noting that FlashAttention is not suitable for fusing GEMM operator chains, while BOLT cannot be applied to the fusion of self-attention modules. The high-performance fused kernel generated by MCFuser validates the effectiveness of our system designs.

$A_{1.2}$**:** ***MCFuser+Relay*** and ***MCFuser+Ansor*** should outperform both BOLT and their respective baselines (Relay and Ansor). These experiments validate that MCFuser is compatible with other works, such as Ansor and Relay, demonstrating the performance of deploying MCFuser on end-to-end models.

$A_{1.3}$**:** The results contains both tuning time of sub-graph modules and end-to-end models. As for the sub-graph modules, the tuning time of the MCFuser should be shorter than that of BOLT and Ansor. In the end-to-end experiments, MCFuser+Relay surpasses BOLT in speed and trades off a minor speed decrease (approximately one minute) compared to Relay for its performance gains. Besides, MCFuser+Ansor offers a clear speed advantage over Ansor. These results have verified MCFuser's efficiency in deployment.

Expected Reproduction Time (in Minutes)

On an A100 GPU, we estimate the total computational time for this artifact to be approximately 2700 minutes. This includes 1500 minutes for $A_{1.1}$, 1200 minutes for $A_{1.2}$, and a neglected duration for $A_{1.3}$. This neglected time for A1.3 is due to the necessary data already being collected during experiments in $A_{1.1}$ and $A_{1.2}$.

Artifact Setup (incl. Inputs)

Hardware: We conducted experiments of $A_{1.1}$ on two servers. One is equipped with one NVIDIA A100-PCIE with 40GB, and the other owns one NVIDIA GeForce RTX 3080 GPU 72-core Intel(R) Xeon(R) Gold 6240C CPU clocked at 2.60GHz. The experiments of $A_{1.2}$ are conducted on the first A100 server.

Software: The compilers and libraries utilized by MCFuser and other baselines include TVM 0.14, Triton 2.1[1], FlashAttention[2], and PyTorch 2.1[3]. The source code of TVM integrated with MCFuser can be found in the *code* directory of the artifact.

Datasets / Inputs: $A_{1.1}$ and $A_{1.3}$ require no additional data downloads. And the model configurations for $A_{1.2}$ have been downloaded in the artifacts by executing the script `artifacts/scripts/e2e/download_data.sh` within artifacts A_1."

Installation and Deployment: First, unpack the artifact from Zenodo, then execute the script `src/install.sh`. The detail is listed as follows/

```
# install dependency library: triton
git clone https://github.com/openai/triton.git
cd triton
git checkout -b mc 0554706f0
python setup.py install

# build tvm + mcfuser
cd src/tvm-mcfuser
mkdir build && cd build
cp ../config.cmake ./
cmake .. -G Ninja && ninja
cd ../python
pip setup.py install
```

Artifact Execution

The workflow of all artifacts contains two tasks: T_1 and T_2. Because we use synthetic input data for all experiments, no dataset has to be generated by tasks. T_1 is the computational task that produces the raw experimental data, and T_2 produces the final results (e.g., plots, tables, etc.) The dependencies between the tasks $T_1 \rightarrow T_2$.

$A_{1.1}$,T_1: The benchmark script for $A_{1.1}$ is `ae/exp_sub_graph.sh`. After executing the script, experimental data will be collected in `graph-{task-name}-{gpu-name}.json`, where `task-name` is `mlp` or `selfatten`, corresponding to the experiments on the fusion of sub-graphs, i.e., GEMM operator chains and self-attention modules, respectively. The parameter `gpu-name` is automatically identified in MCFuser.

$A_{1.1}$,T_2: Fig. 8 can be found under the directory `ae/results/` by executing `python ae/plots/fig8.py`.

$A_{1.2}$,T_1: The measured data is collected by executing `ae/e2e.sh` and will be stored in `e2e-{model-name}-{gpu-name}.json`, where the launch parameter `model-name` have been specified in `ae/e2e.sh` and `gpu-name` is automatically fetched by calling `nvidia-smi`.

[1]https://github.com/openai/triton/tree/0554706f0

[2]https://github.com/Dao-AILab/flash-attention/tree/57ee618

[3]https://pytorch.org/

$A_{1.2}$,T_2: Fig. 9 can be obtained by executing `ae/plots/fig9.py` in `ae/results`.

$A_{1.3}$,T_1: All needed data in $A_{1.3}$ has been produced in the experiments of $A_{1.1}$ and $A_{1.2}$.

$A_{1.3}$,T_2: The data in Table IV can be obtained in `ae/results/tuning_time_graph.csv` and `ae/results/tuning_time_e2e.csv` after executing `python ae/plots/table4.py`.

Artifact Analysis (incl. Outputs)

$A_{1.1}$: In this artifact, we can find that MCFuser outperforms significantly better that Pytorch, Ansor, BoLT and FlashAttention. The speedup compared to PyTorch is attributed to fusing the memory-bound batch GEMMs and reducing off-chip memory access. Ansor adopts declarative loop-oriented scheduling but suffers from inefficient schedule space and lengthy tuning time. In some cases, Ansor fails to fuse the GEMM chains; this is because it concentrates more on the optimization of general operators, leading to sub-optimal performance offusing MBCI operators. BOLT utilizes Cutlass, the state-of-the-art open-source DNN template library for GPUs, to generate fused kernels. However, its flexibility is limited as the templates are manually developed by experts (e.g., the tiling pattern and block execution order are fixed). Moreover, BOLT performs worse in some extreme cases of large tensor size. FlashAttention employs a clever design for fusing self-attention modules, but its implementation suffers from inflexibility. Compared to these works, we design a complete design space for fusing MBCI operators in a more general and efficient way.

$A_{1.2}$: Our experiments demonstrate that *MCFuser+Relay* outperforms both Relay and BOLT, although it falls slightly short of Ansor's performance. This is due to Ansor's extensive tuning for non-MBCI operators, which MCFuser cannot currently handle. However, *MCFuser+Ansor* surpasses Ansor consistently, demonstrating the benefits of combining MCFuser's MBCI operator fusion capabilities with Ansor's strengths in tuning general operators.

$A_{1.3}$: Our experiments demonstrate MCFuser's superior tuning efficiency compared to compiler frameworks like Ansor and BOLT. When tuning sub-graph modules, MCFuser incurs significantly less tuning time overhead. This acceleration is due to MCFuser's pruned design space and analytical performance model, which eliminate the time-consuming hardware profiling and model training steps required by Ansor. For end-to-end networks, our first variant, *MCFuser+Relay*, significantly outperforms both Relay and BOLT in $A_{1.2}$. This improvement comes at an acceptable time cost compared to Relay and a substantial tuning time reduction compared to BOLT. The other variant, *MCFuser+Ansor*, not only outperforms other baselines in $A_{1.2}$ but also achieves a predominant tuning time reduction compared to Ansor, and the operators handled by Ansor account for the majority of *MCFuser+Ansor*'s tuning time.

Artifact Evaluation (AE)

All source codes and artifacts can be found at https://doi.org/10.5281/zenodo.10971908.

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

First, unpack the artifact from Zenodo, then execute the script `src/install.sh`. The detail is listed as follows/

```
# install dependency library: triton
git clone https://github.com/openai/
triton.git
cd triton
git checkout -b mc 0554706f0
python setup.py install

# build tvm + mcfuser
cd src/tvm-mcfuser
mkdir build && cd build
cp ../config.cmake ./
cmake .. -G Ninja && ninja
cd ../python
pip setup.py install
```

Artifact Execution

- To obtain Fig. 8, executing `ae/exp_sub_graph.sh` script first, the corresponding experimental results will be collected in `graph-{task-name}-{gpu-name}.json` . Finally, Fig. 8 can be found under the directory `ae/results/` by executing `python ae/plots/fig8.py`.
- To obtain Fig.9, executing `ae/e2e.sh` script first, the corresponding experimental results will be collected in `e2e-{task-name}-{gpu-name}.json` . Finally, Fig.9 can be found under the directory `ae/results/` by executing `python ae/plots/fig9.py`.
- All needed data in Table IV has been produced after executing the above two artifacts. The cleaned data in Table IV can be obtained in `ae/results/tuning_time_graph.csv` and `ae/results/tuning_time_e2e.csv` after executing python `ae/plots/table4.py`

Artifact Analysis (incl. Outputs)

Fig.8: In this demonstration, MCFuser exhibits a substantial performance advantage over PyTorch, Ansor, BoLT, and FlashAttention, in the execution of both GEMM and Self-attention components. The notable speed enhancements relative to PyTorch affirm the efficacy of our approach in fusing memory-bound batch GEMMs and curtailing extraneous off-chip memory accesses. Ansor fails to fuse GEMM operators in some cases, owing to its emphasis on generic operator optimization. BoLT, while effective in many scenarios, encounters limitations under conditions of exceedingly large tensor dimensions, leading to diminished performance. FlashAttention, with its ingeniously crafted design for merging self-attention mechanisms, is nonetheless hindered by an inherent rigidity in its implementation. In contrast, our work pioneers a comprehensive and meticulously tailored design space, enabling the fusion of MBCI operators in a manner that is not only more versatile but also significantly more efficient.

Fig.9: Our experiments demonstrate that *MCFuser+Relay* outperforms both Relay and BOLT, although it falls slightly short of Ansor's performance. This is due to Ansor's extensive tuning for non-MBCI operators, which MCFuser cannot currently handle. Besides, *MCFuser+Ansor* surpasses Ansor consistently, demonstrating the benefits of combining MCFuser's MBCI operator fusion capabilities with Ansor's strengths in tuning general operators.

Table IV: Our experiments demonstrate MCFuser's superior tuning efficiency compared to compiler frameworks like Ansor and BOLT. When tuning sub-graph modules, MCFuser incurs significantly less tuning time overhead. This acceleration is due to MCFuser's pruned design space and analytical performance model, which eliminate the time-consuming hardware profiling and model training steps required by Ansor.